

Lab 7 Design Class Diagram

Supporting tool - Visual Paradigm | 2 hours 30 minutes

Learning objectives

- Identify design classes and typed attributes from the case description and use case specifications.
- Choose relationships and multiplicities, and justify generalization, aggregation or composition where appropriate.
- Assign operations with parameter and return types to the classes responsible for them.
- Review the diagram for consistency with the requirements and use case model.

Course learning outcomes: CLO 2 - Elicit, analyze, prioritize and document requirements for a software system. CLO 6 - Design, implement, test and present a software system in a group.

Class Diagram

A design class diagram shows the system's classes, typed attributes, operations and relationships. Classes describe kinds of objects; operations allocate the responsibilities identified in the use case specifications.

This lab helps you prepare the design class [diagram](#) required for Milestone 1 of your course project.

Use Visual Paradigm to create your class diagram. Add classes, typed attributes, operations and relationships using the notation below.

Name	Symbol	Meaning and direction
Class	Rectangle with compartments	Represents a class. It can contain the class name, attributes, and operations.
Attribute	- attributeName : Type	Data held by each object. "-" is private; "+" is public.
Operation / Method	+ operationName(parameter: Type): ReturnType	Represents a behavior or function provided by a class.
Association	—————	Shows a relationship between two classes.
Generalization	—————▷	Shows inheritance. The hollow triangle points to the general/parent class.
Aggregation	◇—————	Shared whole-part relationship; the hollow diamond is at the whole. Use it only when it clarifies the model.
Composition	◆—————	Exclusive whole-part ownership; the filled diamond is at the whole. A part belongs to at most one whole and is deleted with it unless detached.
Multiplicity	1, 0..1, *, 1..*	Shows how many instances of one class can be associated with another.

Part A QU Print Shop - Guided practice

A.1 Identify classes and attributes

Recall the Print Shop: users submit and revise queued jobs, printing runs automatically, and collection may include a receipt. PAS handles credit and charges; staff can request summaries of their own jobs.

Start from the Print Shop use case diagram in [Lab 5](#) and the worked specifications of UC001 Submit print job and UC003 Collect printed document in [Lab 6](#).

The worked table below identifies the classes, typed attributes and supporting evidence.

PAS remains external; PrintShop and PASGateway are design choices for coordinating its services.

Class	Typed attributes	Evidence and justification
User	userId, name, email: String	Each job has an owner; email is an assumed notification channel.
Student	studentId, program	Specialize User; profile details are assumed.
StaffMember	staffId, department: String	Specialize User; profile details are assumed.
PrintJob	jobId: String; submittedAt, printedAt, collectedAt: LocalDateTime; status: JobStatus; cost: BigDecimal	Tracks job state and cost; timestamps are assumed.
Document	documentId, fileName, fileType: String; fileSizeBytes: long; pageCount: int	Stores the uploaded file and assumed metadata.
PrintingOptions	copies: int; colour, doubleSided: boolean	The scenario specifies these three printing choices.
CollectionReceipt	receiptNo: String; issuedAt: LocalDateTime	Optional receipt linked to its collected job.
PrintShop / PASGateway	No additional stored attributes shown	Coordinate use cases and adapt PAS calls; PAS owns balances and charges.
JobStatus	QUEUED, PRINTED, COLLECTED	Enumeration of the three states in the scenario.

A.2 Relationships and multiplicities

One User owns 0..* **PrintJobs**; each job refers to one **Document**, owns one set of **PrintingOptions** and has 0..1 **CollectionReceipts**. Use composition for options owned exclusively by a job. **Student** and **StaffMember** specialize User. These are justified design choices, not a requirement to use every relationship type.

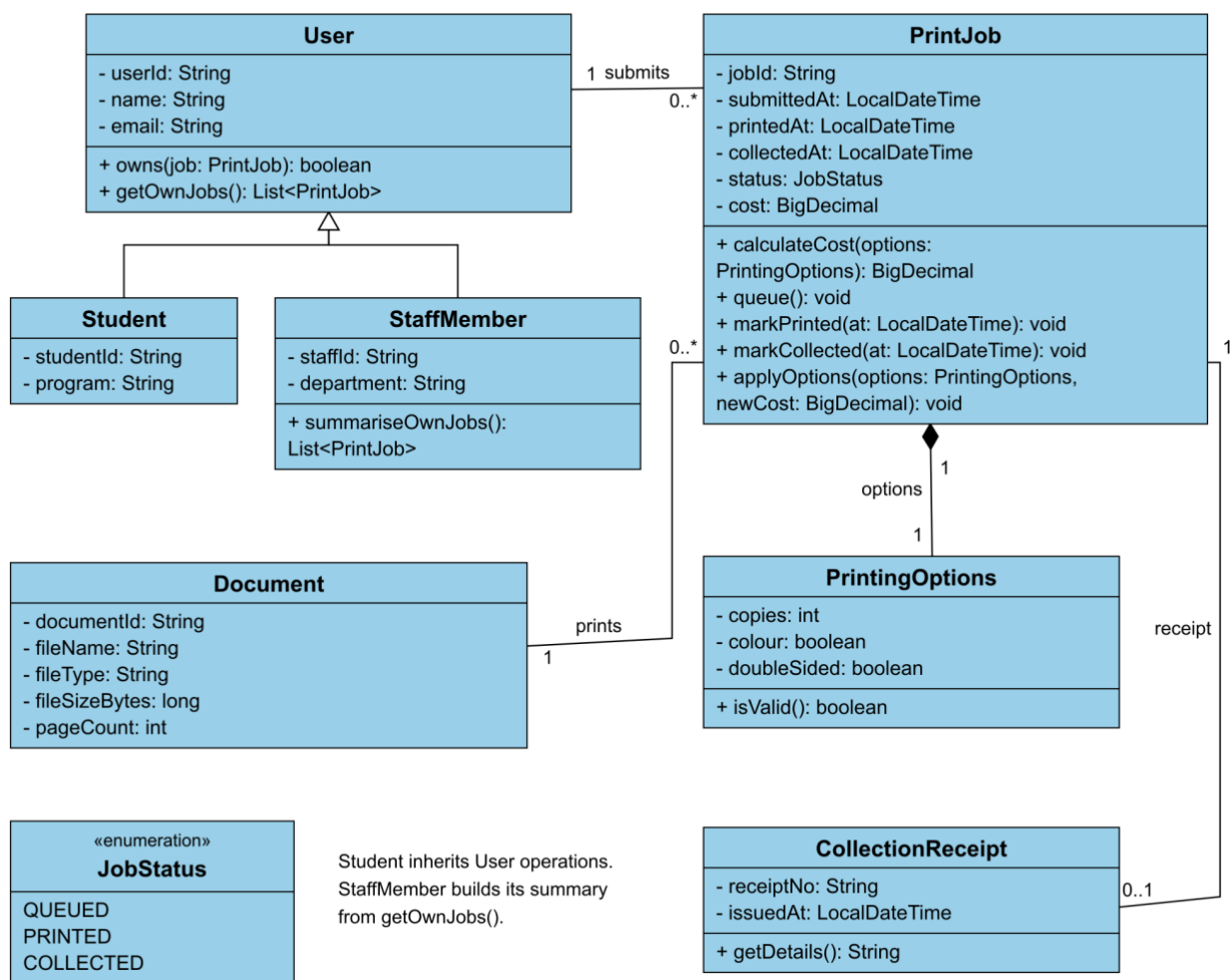
A.3 Assign operations

PrintShop coordinates these calls. User checks job ownership and retrieves its own jobs; **StaffMember** uses those jobs to produce its summary. Student inherits the common operations.

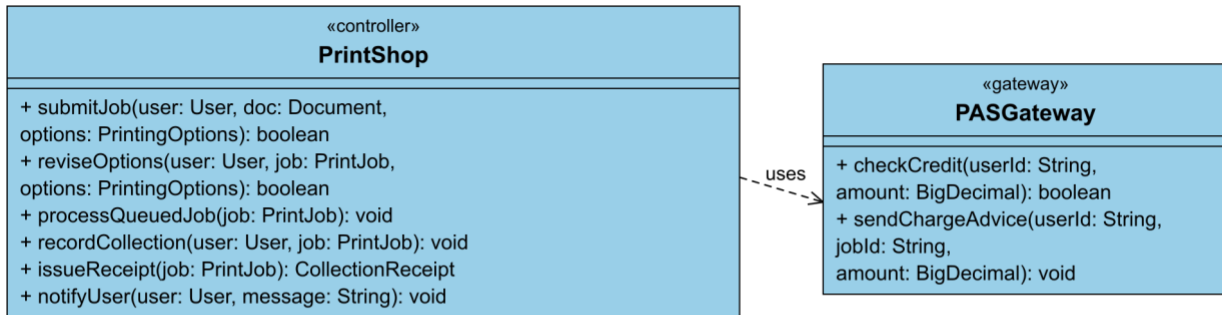
Design assumptions: email notifications, profile details, document metadata and event timestamps enrich this worked example; a document may be reused and a job has at most one receipt. Money and DateTime are application data types. Routine accessors are omitted; not every class needs additional attributes or operations.

Class	Operation with types	Source and placement reason
PrintJob	calculateCost(options: PrintingOptions): BigDecimal	UC001 step 2: costs the document with proposed options.
PASGateway	checkCredit(userId: String, amount: BigDecimal): boolean	UC001 step 2 / UC006: asks PAS for the credit decision.
PrintJob	queue(): void	UC001 step 3: sets the accepted job to QUEUED.
PrintJob	markCollected(at: LocalDateTime): void	UC003 step 2: records collection and its timestamp.
StaffMember	summariseOwnJobs(): List<PrintJob>	Lab 5 staff summary: uses only this member's jobs.

Visual Paradigm Standard (Qatar university)



Coordinating services



Controller calls the domain operations above; PAS remains external.

Accept/revise only after PAS approves the proposed cost; a failed request leaves jobs unchanged.

Revise queued jobs only; collect printed jobs only; issue a requested receipt after collection.

submitJob returns false on rejection. Printing and collection timestamps are set when each event occurs.

Design assumptions and omitted routine accessors are explained in A.3.

Part B All Phones Shop

Individual assessment. Begin with a 10-minute instructor review of the Lab 6 APS diagram and Lab 7 APS Reference Specifications: UC001 Register client, UC003 List item for sale and UC006 Place bid.

B.1 Identify classes and typed attributes - 25 minutes

Use the [APS case from Lab 3](#) and [Lab 6, Figure 2](#) and the [Register client example](#). Keep PoB financial accounts and login outside APS.

Identify classes needed by the case and specifications. Record typed attributes and a brief justification in the table; state assumptions. Add rows as needed.

Class name	Typed attributes	Evidence and justification

B.2 Develop the APS class diagram - 35 minutes

Develop the diagram in Visual Paradigm using B.1. Show typed attributes and visibility, associations with multiplicity at both ends, and justified generalization and whole-part relationships. Briefly explain your main relationship choices below the diagram.

B.3 Assign operations from the use case specifications - 30 minutes

Derive operations from UC001 Register client, UC003 List item for sale and UC006 Place bid. For each, record its responsible class, typed parameters, return type and source use case step. Add the operations to your diagram.

Class	Operation with parameter and return types	Use case step and placement reason

B.4 Review and refine the APS class diagram - 15 minutes

Exchange diagrams with a classmate. Check requirement coverage, operation ownership and types, relationships and multiplicities. Record one strength, one suggested improvement and the change you made.

Submission requirements

Allow 15 minutes. Submit the three files below in a folder named **Lab 7 Class Diagram** in your own repository in the [CMPS310 course organization](#). Follow the commit and push workflow from [Lab 2](#), then check that the files open on GitHub. Part A is guided practice and is not submitted.

File	Required contents
lab07-solution.docx	Name and student ID; completed Part B tables; final APS diagram; relationship justifications; peer feedback and the change made; stated assumptions.
aps-class-diagram.png	Readable export of the final APS class diagram.
aps-class-diagram.vpp	Editable Visual Paradigm project containing the APS class diagram.

Assessment criteria (10 points)

Criterion	Points
Relevant classes and typed attributes, justified from the case and specifications	3.5
Operations with parameter and return types, traced to requirements and assigned to responsible classes	3.5
Correct relationships, multiplicities and visibility; readable diagram consistent with the use case model	3

Further reading

[Visual Paradigm - What is Class Diagram?](#)

[OMG UML 2.5.1 - Clause 9 Classification and Clause 11 Structured Classifiers](#)